UML Components

Part-2

Objectives

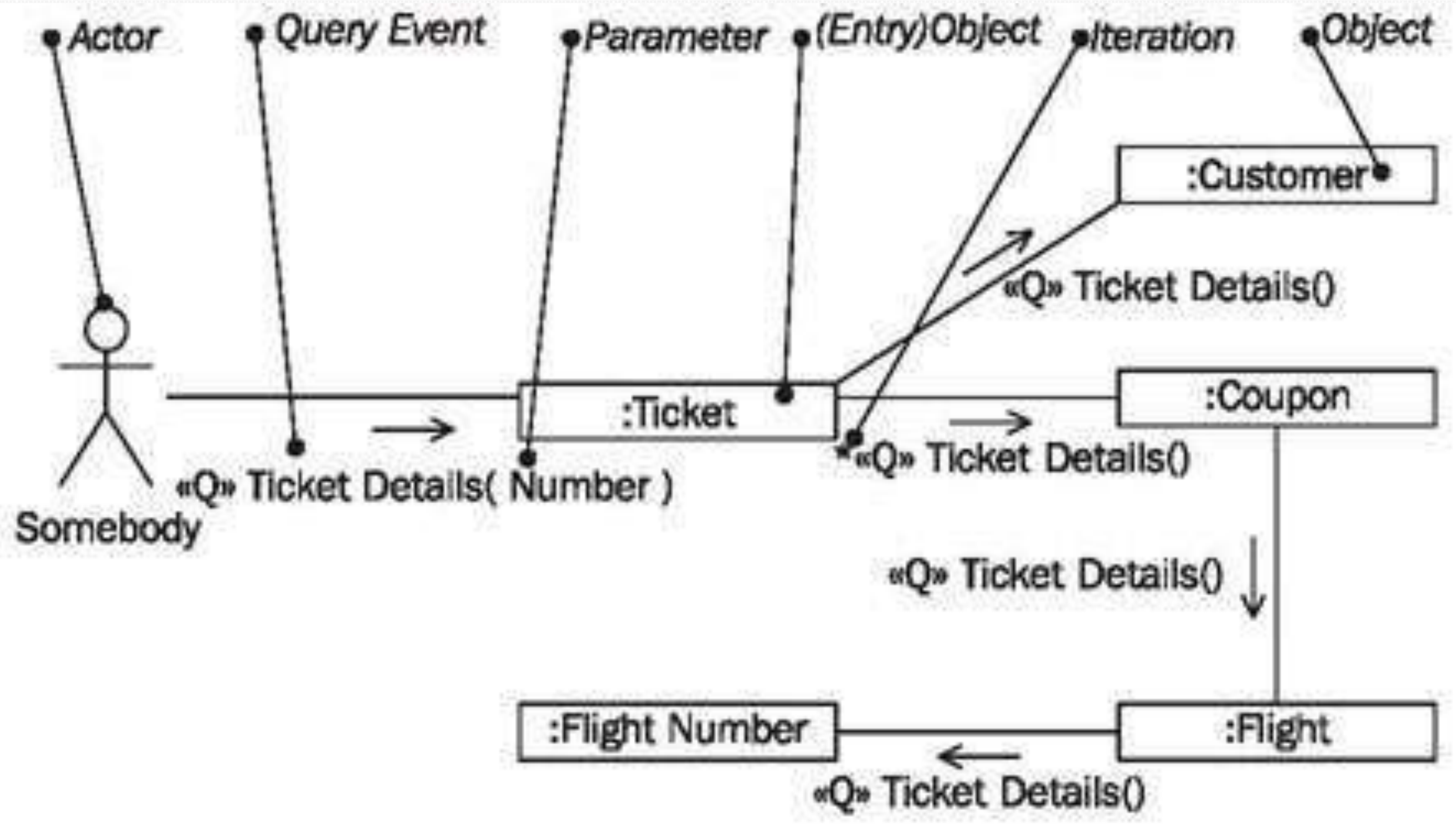
- The runtime components
- The executable components
- Class assignment and interfaces of components

components types

- Component:
  - Physical set of object based or functional construct that provides functionality through well defined communication mechanism.
- Types:
  - **Compile time / Link Time:** Object code libraries.
  - **Run-time**: in-memory instantiation of these build time constructs.
  - **Executable:** Example is building an application to perform a count of statements in the source code files:

    cat *.cpp |grep ";"| wc –l
    Here pipe provides a data port to the components for communication.

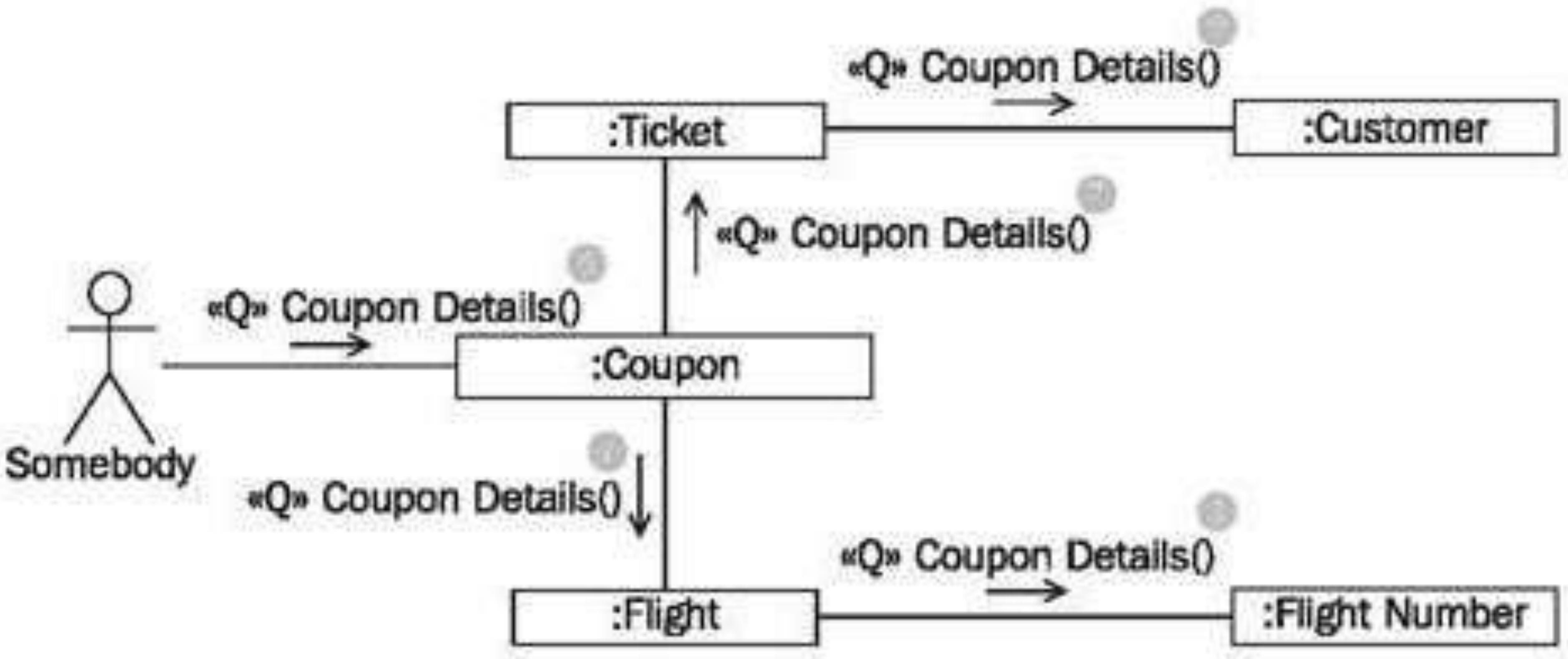
Communication Diagram



Communication Diagram

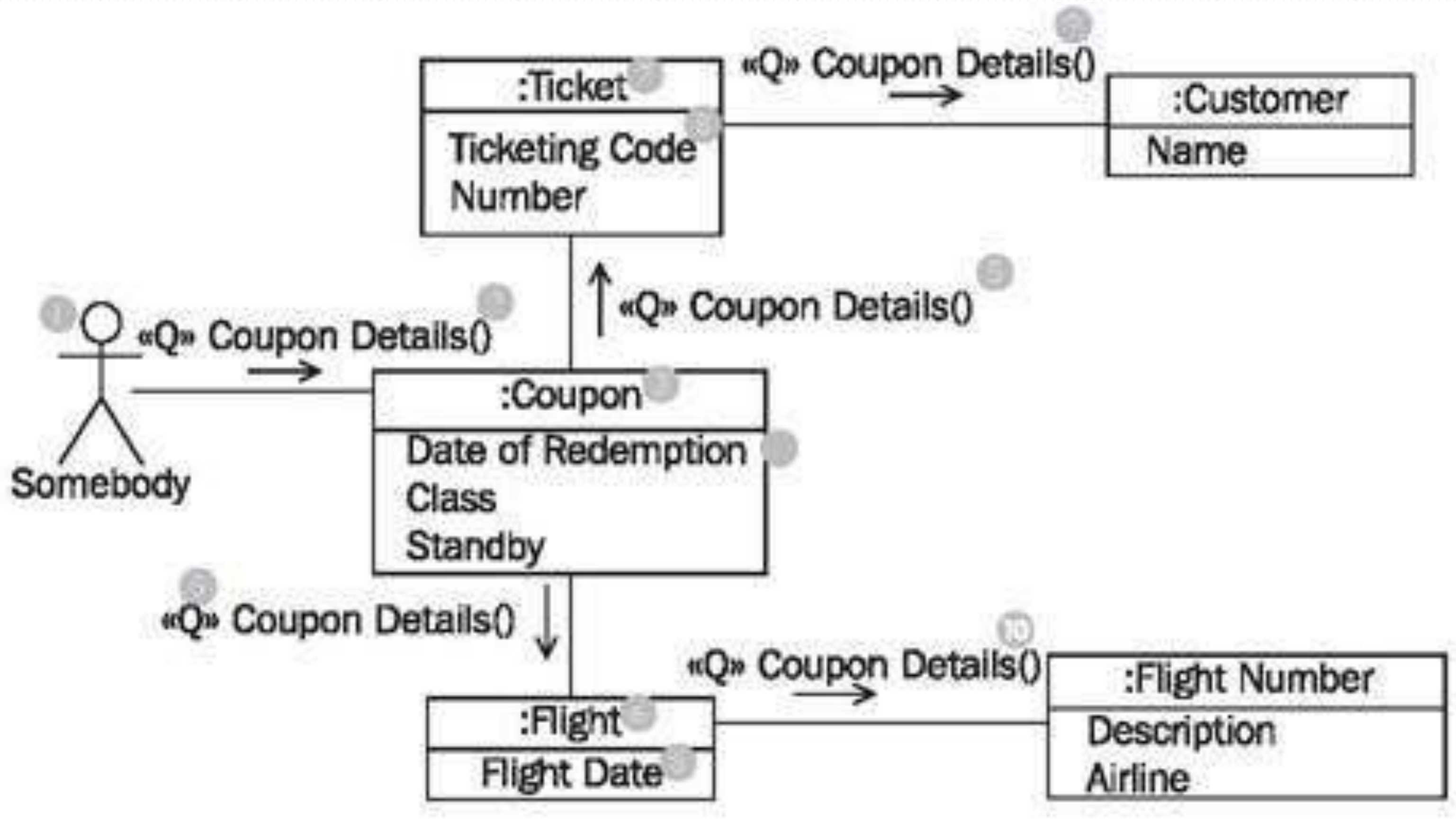
- **Query Event**
- A query event represents a query for information:
- **Parameter**
- Parameters allow for attaching information to an event, e.g., the number of a ticket, so that the correct ticket can be read:

Communication Diagram



Communication Diagram with out Attributes

Communication Diagram

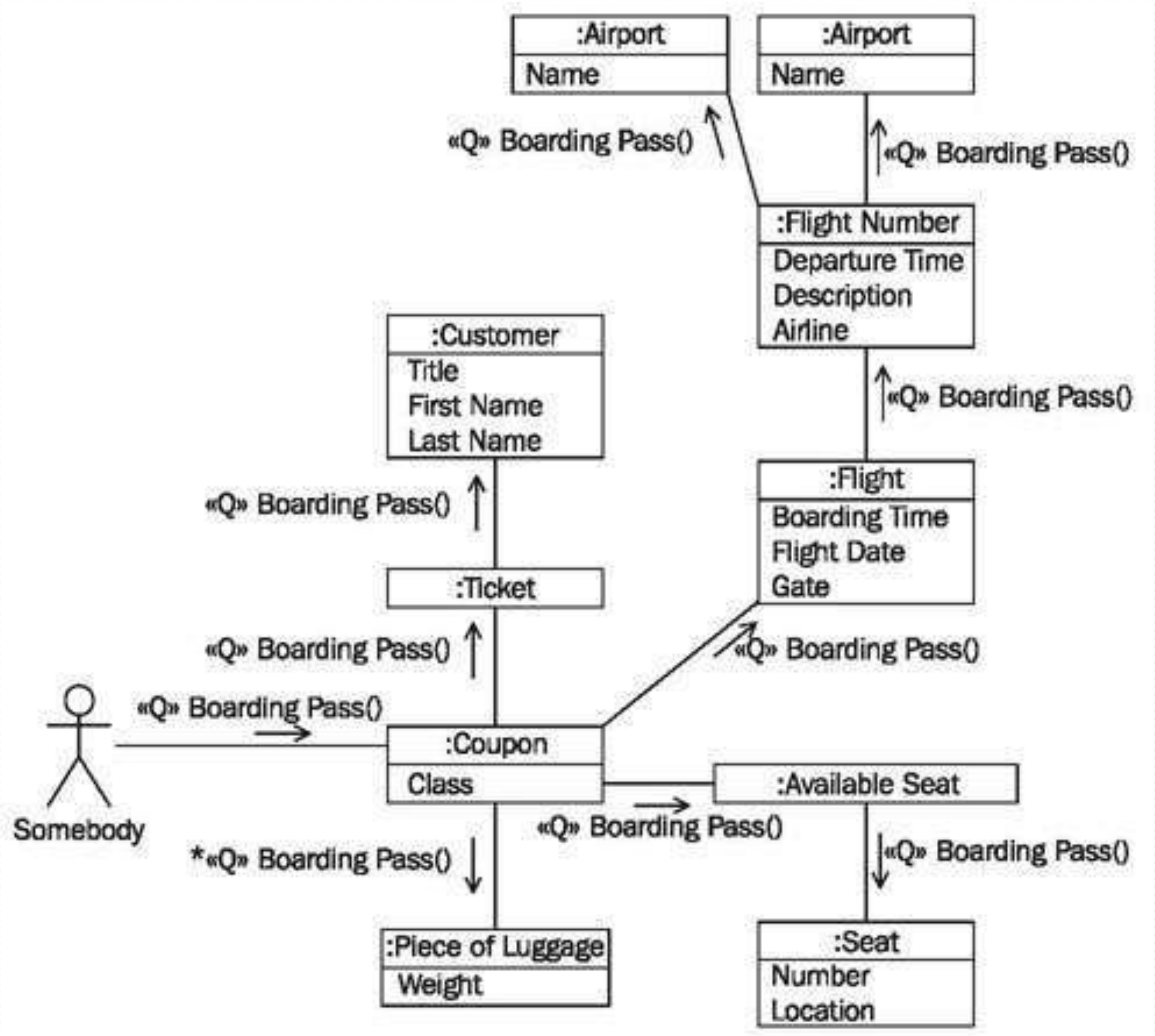


Communication Diagram with Attributes

Constructing Communication Diagrams

- **Checklist for Communication Diagrams:**
- Draft query result—What do we want?
- Identify involved classes—Which classes do we need?
- Define initial object—Where do we start?
- Design event path—Where do we go?
- Amend event path—Exactly which objects do we need?
- Identify necessary attributes—What exactly do we want to know?
- Verify the communication diagram—Is everything correct?

Communication Diagram

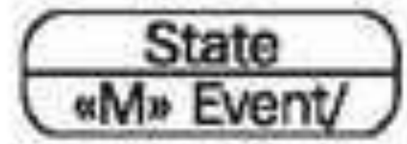


State-chart diagrams

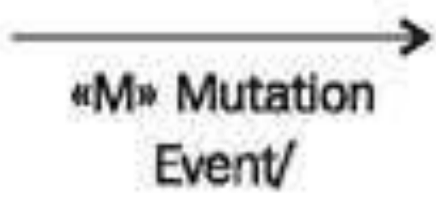
- **State:** The state of an object is always determined by its attributes and associations. States in statechart diagrams represent a *set* of those value combinations, in which an object *behaves the same* in response to events.
- Therefore, not every modification of an attribute leads to a new state.
- **Transition:** A transition represents the change from one state to another.

State-chart diagrams

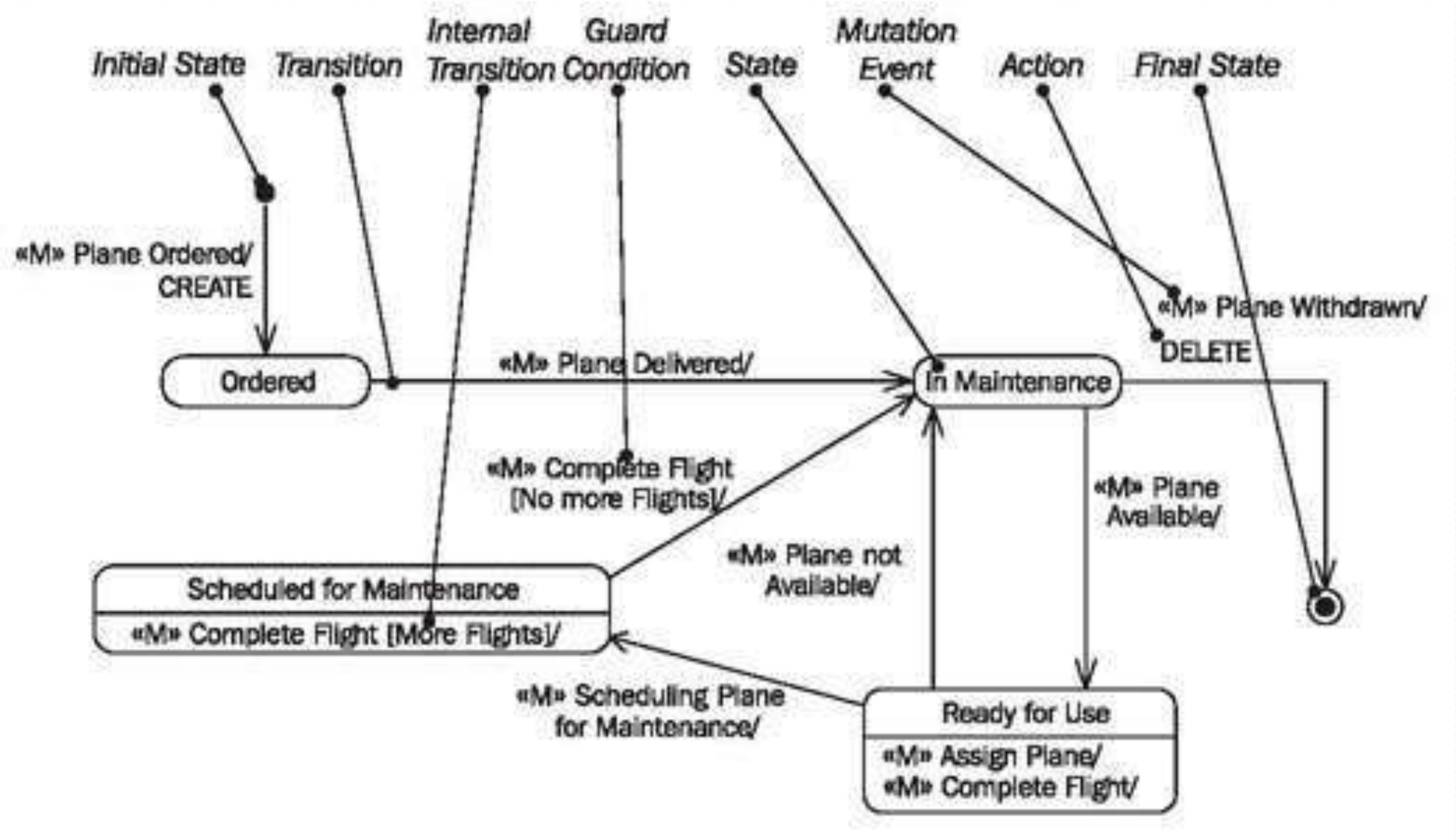
- **Internal Transition:** transition from one state to itself. Object handles event without changing its state.



- **Mutation Event:** The initiator of a transition from one state to another, or for an internal transition, where the state remains the same.

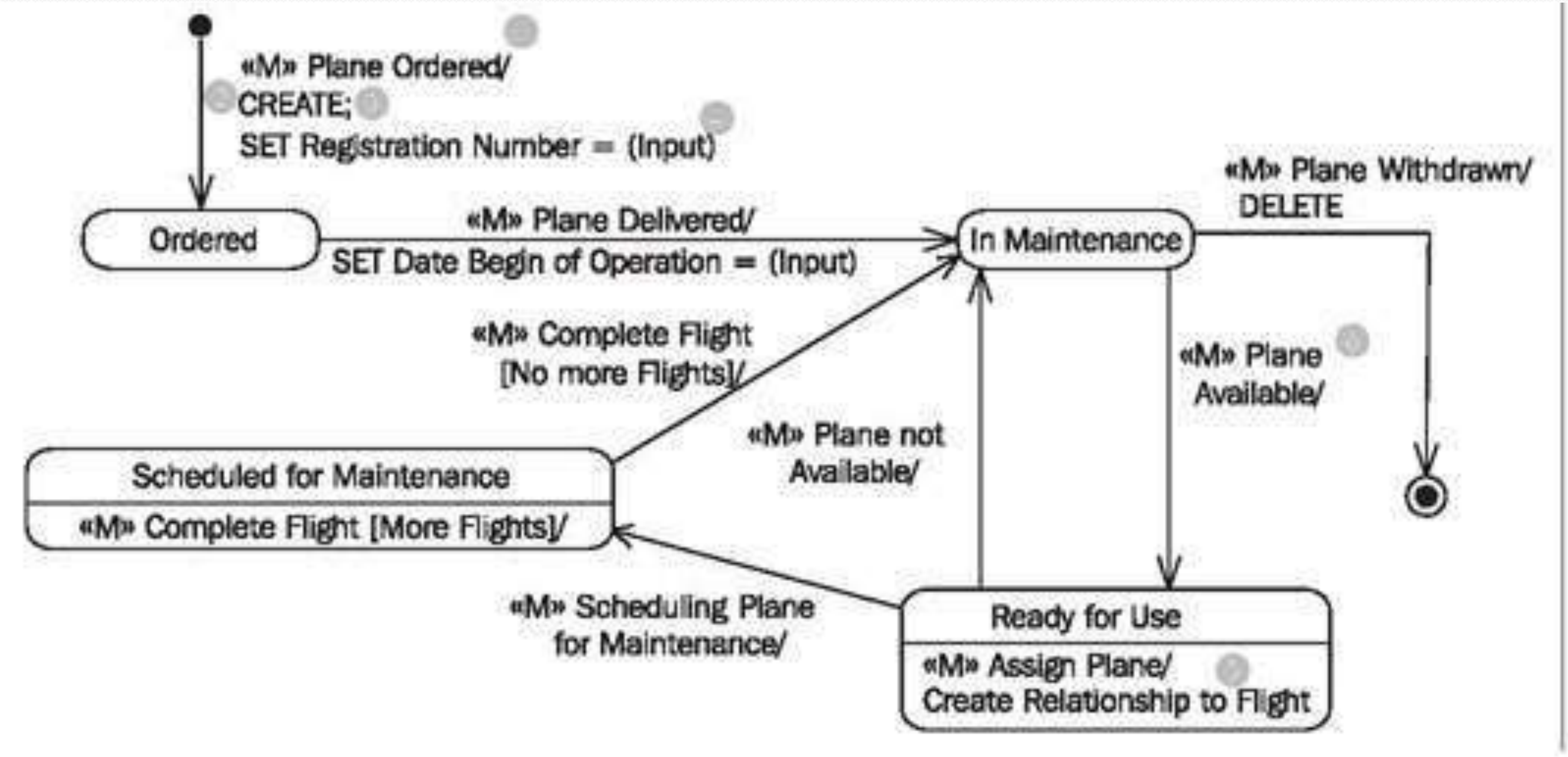


State-chart diagrams



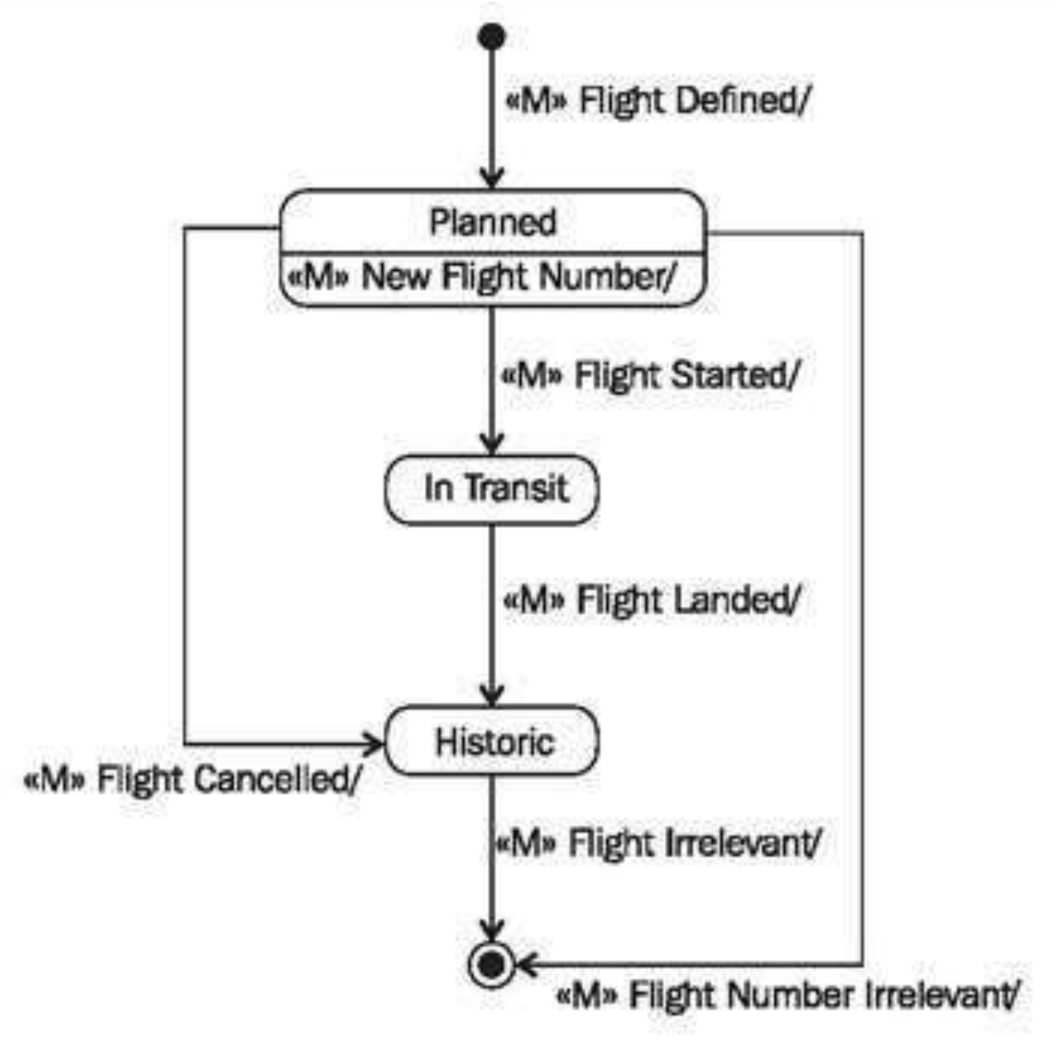
Elements of the statechart diagram

State-chart diagrams



Statechart diagram with internal transitions and guard conditions.

State-chart diagrams



Statechart diagram of the class "Flight"

State-chart diagrams

- **Checklist : Statechart Diagrams**
- Identify mutation events relevant for the object—What affects the object?
- Group relevant events chronologically—How does a normal life look?
- Model states and transitions—Which states are there?
- Add actions to the statechart diagram—What do objects do?
- Verify the statechart diagram—Is everything correct?